some of the different types of tag helpers available within ASP.NET Core and how to use them.

Finally, you will see how the `PageModel` object acts as a *page controller*, handling the request for a page and deciding which model and view to use. Handler methods play a significant role in the request processing, and you will learn the important conventions behind their use and the return types, or `IActionResult`s, they commonly work with.

3.1 Working with Razor syntax

All web development frameworks need to be able to generate HTML dynamically. Almost exclusively, they rely on a design pattern called *Template View*. This pattern involves using markers or placeholders consisting of server-side code embedded in HTML that resolve to calls to process and render dynamic content.

Dynamic content can take many forms. Often it will take the form of data extracted from a data store, such as a database, but as you have seen, it can also simply be the result of some computation, such as the time of day. In addition to the content itself, you will need to embed server-side code to control the presentation of the content. For example, if the dynamic content is a collection, such as a list, you will need to iterate over it in code to display each item. Or you might need to only show the data under certain conditions, such as if the user has permission to see it. So in addition to enabling you to embed instances of data into a page, the templating syntax must also enable you to include statement blocks that control processing within the page.

Many frameworks, such as older versions of ASP.NET, PHP, Ruby on Rails, and Java Server Pages, use HTML-like tags as markers to let the template processor know where the transitions are between HTML and server-side code. Razor syntax uses the `@` symbol as a transition marker and has several simple rules. The first rule is that Razor syntax is only applicable to the contents of Razor files that have a .cshtml extension. That's an amalgam of *cs* for C Sharp and *html* for Hypertext